

10 — Database Design Patterns

Table of Contents

1. [Learning Objectives](#)
 2. [Soft Deletes](#)
 3. [Audit Trails](#)
 4. [Row Versioning / Temporal Tables](#)
 5. [Polymorphic Associations](#)
 6. [Entity-Attribute-Value \(EAV\)](#)
 7. [Hierarchies and Trees](#)
 8. [Optimistic Locking](#)
 9. [Outbox Pattern](#)
 10. [State Machine Pattern](#)
 11. [SQL Examples](#)
 12. [Common Mistakes](#)
 13. [Best Practices](#)
 14. [Performance Considerations](#)
 15. [Interview Questions & Answers](#)
 16. [Exercises with Solutions](#)
 17. [Cross-References](#)
-

Learning Objectives

After completing this section you will be able to:

- Implement soft deletes without breaking application queries
 - Build a comprehensive audit trail system
 - Handle entity versioning for temporal and historical data
 - Design polymorphic associations safely
 - Avoid the EAV antipattern while knowing when a controlled version is acceptable
 - Implement optimistic locking for concurrent updates
-

Soft Deletes

Pattern

Instead of physically deleting rows, mark them as deleted with a timestamp.

```
-- Basic soft delete column
ALTER TABLE posts ADD COLUMN deleted_at TIMESTAMPTZ;
-- NULL = not deleted, timestamp = when it was deleted

-- Soft delete
UPDATE posts SET deleted_at = now() WHERE post_id = 42;

-- Query active records only
SELECT * FROM posts WHERE deleted_at IS NULL;

-- Query deleted records
```

```
SELECT * FROM posts WHERE deleted_at IS NOT NULL;

-- Restore
UPDATE posts SET deleted_at = NULL WHERE post_id = 42;
```

Partial index for performance

```
-- Index only active records (much smaller, faster)
CREATE INDEX idx_posts_active ON posts (user_id, created_at DESC)
    WHERE deleted_at IS NULL;

-- Unique constraint on active records only
CREATE UNIQUE INDEX uq_users_email_active ON users (email)
    WHERE deleted_at IS NULL;

-- Allows: alice@example.com (active) + alice@example.com (deleted, archived)
```

Cascading soft deletes

```
-- When deleting a post, also soft-delete its comments
CREATE OR REPLACE FUNCTION soft_delete_post(p_post_id BIGINT)
RETURNS void LANGUAGE plpgsql AS $$
BEGIN
    UPDATE posts SET deleted_at = now() WHERE post_id = p_post_id AND deleted_at
IS NULL;
    UPDATE comments SET deleted_at = now() WHERE post_id = p_post_id AND deleted_at
IS NULL;
END;
$$;
```

Trade-offs

Soft Delete Pros:

- ✓ Data recovery possible
- ✓ Audit trail preserved
- ✓ No FK cascade issues
- ✓ Legal/compliance data retention

Soft Delete Cons:

- ✗ All queries need WHERE deleted_at IS NULL
- ✗ Unique indexes need partial index workaround
- ✗ Table grows indefinitely (archive old data)
- ✗ Joins may accidentally include deleted rows

Audit Trails

Pattern 1: Separate audit log table

```

-- Generic audit log
CREATE TABLE audit_log (
    audit_id      BIGINT      GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    table_name    TEXT        NOT NULL,
    operation     TEXT        NOT NULL CHECK (operation IN
('INSERT', 'UPDATE', 'DELETE')),
    record_id     TEXT        NOT NULL, -- PK as text (generic)
    old_data      JSONB,
    new_data      JSONB,
    changed_by    TEXT        NOT NULL DEFAULT current_user,
    changed_at    TIMESTAMPTZ NOT NULL DEFAULT now(),
    ip_address    INET,
    app_user_id   BIGINT      -- application-level user (not DB user)
);

CREATE INDEX idx_audit_table_record ON audit_log (table_name, record_id);
CREATE INDEX idx_audit_changed_at   ON audit_log (changed_at DESC);
CREATE INDEX idx_audit_app_user     ON audit_log (app_user_id);

-- Trigger function (one function, reused across tables)
CREATE OR REPLACE FUNCTION log_changes()
RETURNS TRIGGER LANGUAGE plpgsql AS $$
BEGIN
    INSERT INTO audit_log (table_name, operation, record_id, old_data, new_data)
    VALUES (
        TG_TABLE_NAME,
        TG_OP,
        CASE TG_OP
            WHEN 'DELETE' THEN OLD.id::TEXT
            ELSE NEW.id::TEXT
        END,
        CASE TG_OP
            WHEN 'INSERT' THEN NULL
            ELSE to_jsonb(OLD)
        END,
        CASE TG_OP
            WHEN 'DELETE' THEN NULL
            ELSE to_jsonb(NEW)
        END
    );
    RETURN NULL; -- AFTER trigger, return value ignored
END;
$$;

-- Apply to any table
CREATE TRIGGER audit_users
    AFTER INSERT OR UPDATE OR DELETE ON users
    FOR EACH ROW EXECUTE FUNCTION log_changes();

CREATE TRIGGER audit_orders

```

```
AFTER INSERT OR UPDATE OR DELETE ON orders
FOR EACH ROW EXECUTE FUNCTION log_changes();
```

Pattern 2: History table (shadow table)

```
-- For each table, maintain a history copy with valid_from/valid_to
CREATE TABLE products_history (
    history_id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    product_id INTEGER NOT NULL,
    -- All columns from products table...
    name TEXT,
    price NUMERIC(12,4),
    -- Temporal columns
    valid_from TIMESTAMPTZ NOT NULL,
    valid_to TIMESTAMPTZ, -- NULL = current record
    operation CHAR(1) NOT NULL CHECK (operation IN ('I','U','D')),
    changed_by TEXT NOT NULL DEFAULT current_user
);

CREATE INDEX idx_product_history_id ON products_history (product_id, valid_from
DESC);
CREATE INDEX idx_product_history_curr ON products_history (product_id) WHERE
valid_to IS NULL;

-- Trigger to maintain history
CREATE OR REPLACE FUNCTION maintain_product_history()
RETURNS TRIGGER LANGUAGE plpgsql AS $$
BEGIN
    IF TG_OP = 'INSERT' THEN
        INSERT INTO products_history (product_id, name, price, valid_from,
operation)
        VALUES (NEW.product_id, NEW.name, NEW.price, now(), 'I');

    ELSIF TG_OP = 'UPDATE' THEN
        -- Close the previous version
        UPDATE products_history SET valid_to = now()
        WHERE product_id = OLD.product_id AND valid_to IS NULL;
        -- Insert new version
        INSERT INTO products_history (product_id, name, price, valid_from,
operation)
        VALUES (NEW.product_id, NEW.name, NEW.price, now(), 'U');

    ELSIF TG_OP = 'DELETE' THEN
        UPDATE products_history SET valid_to = now()
        WHERE product_id = OLD.product_id AND valid_to IS NULL;
        INSERT INTO products_history (product_id, name, price, valid_from, valid_to,
operation)
        VALUES (OLD.product_id, OLD.name, OLD.price, now(), now(), 'D');
    END IF;
    RETURN NULL;
```

```
END;  
$$;
```

Row Versioning / Temporal Tables

Optimistic concurrency with version number

```
CREATE TABLE documents (  
  doc_id      BIGINT      GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
  title       TEXT        NOT NULL,  
  content     TEXT        NOT NULL,  
  version     INTEGER     NOT NULL DEFAULT 1, -- version counter  
  updated_by  BIGINT      NOT NULL,  
  updated_at  TIMESTAMPTZ NOT NULL DEFAULT now()  
);  
  
-- Update with version check (prevents lost updates)  
UPDATE documents  
SET title = 'New Title', version = version + 1, updated_at = now()  
WHERE doc_id = 42 AND version = 5; -- only succeeds if we have the right version  
  
-- Check if update succeeded  
GET DIAGNOSTICS affected_rows = ROW_COUNT;  
IF affected_rows = 0 THEN  
  RAISE EXCEPTION 'Concurrent modification detected. Please reload and retry.';  
END IF;
```

System-versioned temporal tables

```
CREATE TABLE price_history (  
  price_id      INTEGER      GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
  product_id    INTEGER      NOT NULL REFERENCES products(product_id),  
  price         NUMERIC(12,4) NOT NULL,  
  -- System-time period (when was this the current price in the DB)  
  sys_from      TIMESTAMPTZ  NOT NULL DEFAULT now(),  
  sys_to        TIMESTAMPTZ, -- NULL = current  
  -- Application-time period (when was this the effective price in reality)  
  app_from      DATE          NOT NULL,  
  app_to        DATE          -- NULL = currently effective  
);  
  
-- What was the price of product 42 on Jan 15, 2024?  
SELECT price  
FROM price_history  
WHERE product_id = 42  
  AND app_from <= '2024-01-15'::DATE  
  AND (app_to IS NULL OR app_to > '2024-01-15'::DATE)  
  AND (sys_to IS NULL OR sys_to > '2024-01-15'::TIMESTAMPTZ);
```

Polymorphic Associations

Pattern 1: Separate FK columns (preferred)

```
-- Comments can be on posts OR videos OR photos
CREATE TABLE comments (
  comment_id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  user_id    BIGINT NOT NULL REFERENCES users(user_id),
  -- Polymorphic: exactly one of these should be non-NULL
  post_id    BIGINT REFERENCES posts(post_id),
  video_id   BIGINT REFERENCES videos(video_id),
  photo_id   BIGINT REFERENCES photos(photo_id),
  body       TEXT NOT NULL,
  created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  CONSTRAINT exactly_one_parent CHECK (
    (post_id IS NOT NULL)::INT +
    (video_id IS NOT NULL)::INT +
    (photo_id IS NOT NULL)::INT = 1
  )
);

CREATE INDEX idx_comments_post ON comments (post_id) WHERE post_id IS NOT NULL;
CREATE INDEX idx_comments_video ON comments (video_id) WHERE video_id IS NOT NULL;
CREATE INDEX idx_comments_photo ON comments (photo_id) WHERE photo_id IS NOT NULL;
```

Pattern 2: Generic foreign key (anti-pattern, avoid)

```
-- BAD: no FK integrity, no index without workaround
CREATE TABLE comments (
  comment_id BIGINT PRIMARY KEY,
  entity_type TEXT NOT NULL, -- 'post', 'video', 'photo'
  entity_id   BIGINT NOT NULL, -- can't have FK here!
  body        TEXT NOT NULL
  -- NO referential integrity guaranteed!
);
```

Pattern 3: Table inheritance (if truly polymorphic)

```
-- Base entity table
CREATE TABLE entities (
  entity_id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  entity_type TEXT NOT NULL CHECK (entity_type IN ('post','video','photo')),
  created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE posts (LIKE entities INCLUDING ALL, title TEXT, body TEXT);
CREATE TABLE videos (LIKE entities INCLUDING ALL, url TEXT, duration_ms INTEGER);
```

```
-- Comments reference the base entity
CREATE TABLE comments (
  comment_id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  entity_id  BIGINT NOT NULL REFERENCES entities(entity_id) ON DELETE CASCADE,
  body       TEXT   NOT NULL
);
```

Entity-Attribute-Value (EAV)

EAV stores arbitrary attributes as rows instead of columns. Generally an **antipattern**, but acceptable for specific use cases.

Why EAV is bad

EAV table:

entity_id	attribute	value
1	color	red
1	size	Large
1	weight	2.5
2	color	blue

← stored as TEXT!

Problems:

- No type safety (weight is TEXT, not NUMERIC)
- No constraints (negative weight allowed)
- Terrible query performance (pivot needed for multi-attr query)
- No joins on attribute values
- Difficult to index effectively

When EAV is acceptable (controlled version)

```
-- Controlled EAV with JSONB (better than string EAV)
CREATE TABLE product_attributes (
  product_id INTEGER NOT NULL REFERENCES products(product_id),
  attributes JSONB NOT NULL DEFAULT '{}',
  -- Validated by application; GIN indexed for search
);

-- Even better: typed EAV
CREATE TABLE product_numeric_attrs (
  product_id INTEGER NOT NULL REFERENCES products(product_id),
  attr_name  TEXT NOT NULL,
  attr_value NUMERIC NOT NULL,
  PRIMARY KEY (product_id, attr_name)
);

CREATE TABLE product_text_attrs (
  product_id INTEGER NOT NULL REFERENCES products(product_id),
```

```

    attr_name    TEXT        NOT NULL,
    attr_value   TEXT        NOT NULL,
    PRIMARY KEY (product_id, attr_name)
);
-- Separate tables per type provide basic type safety

```

Hierarchies and Trees

Adjacency List (reviewed)

```

-- Simple, good for writes, recursive CTE for reads
parent_id INTEGER REFERENCES self(id)

-- Get subtree (PostgreSQL 14+ has CYCLE detection built-in)
WITH RECURSIVE tree AS (
    SELECT id, name, parent_id, ARRAY[id] AS path, false AS is_cycle
    FROM nodes WHERE id = :root
    UNION ALL
    SELECT n.id, n.name, n.parent_id, t.path || n.id,
           n.id = ANY(t.path) -- cycle detection
    FROM nodes n, tree t
    WHERE n.parent_id = t.id AND NOT is_cycle
)
SELECT * FROM tree;

```

Materialized Path

```

CREATE TABLE categories (
    id        INTEGER PRIMARY KEY,
    name      TEXT        NOT NULL,
    path      TEXT        NOT NULL -- '1.5.12.' format
);
CREATE INDEX idx_cat_path ON categories (path);

-- Subtree query: O(1) with index!
SELECT * FROM categories WHERE path LIKE '1.5.%';

-- Parent lookup
SELECT * FROM categories WHERE '1.5.12.' LIKE path || '%';

```

ltree extension (recommended)

```

CREATE EXTENSION IF NOT EXISTS ltree;

CREATE TABLE categories (
    id        INTEGER PRIMARY KEY,
    name      TEXT        NOT NULL,
    path      LTREE       NOT NULL

```

```

);

CREATE INDEX idx_cat_ltree ON categories USING GIST (path);

-- Descendants of node '1.5'
SELECT * FROM categories WHERE path <@ '1.5';

-- Ancestors of node '1.5.12'
SELECT * FROM categories WHERE path @> '1.5.12';

-- Children (direct)
SELECT * FROM categories WHERE path ~ '1.5.*{1}';

```

Optimistic Locking

```

-- Version-based optimistic lock
CREATE TABLE reservations (
    reservation_id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    seat_id        INTEGER NOT NULL,
    user_id        INTEGER NOT NULL,
    status         TEXT     NOT NULL DEFAULT 'pending',
    version        INTEGER NOT NULL DEFAULT 1,
    updated_at     TIMESTAMPTZ NOT NULL DEFAULT now()
);

-- Application workflow:
-- 1. Read: SELECT * FROM reservations WHERE id = :id
-- 2. Modify in memory
-- 3. Update with version check:
UPDATE reservations
SET status = 'confirmed',
    version = version + 1,
    updated_at = now()
WHERE reservation_id = :id
    AND version = :read_version; -- optimistic lock check

-- If 0 rows affected: another process modified it → retry
-- If 1 row affected: success

```

Outbox Pattern

Ensures that database state changes and downstream messages are in sync (transactional consistency for event-driven systems).

```

-- Outbox table: events to be published
CREATE TABLE outbox (
    outbox_id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    event_type TEXT     NOT NULL,

```

```

    payload      JSONB      NOT NULL,
    status       TEXT       NOT NULL DEFAULT 'pending'
                    CHECK (status IN ('pending','processing','published','failed')),
    attempts     INTEGER NOT NULL DEFAULT 0,
    created_at   TIMESTAMPTZ NOT NULL DEFAULT now(),
    processed_at TIMESTAMPTZ
);

CREATE INDEX idx_outbox_pending ON outbox (created_at)
    WHERE status = 'pending';

-- Application: within the same transaction as the business operation
BEGIN;
-- Business operation
INSERT INTO orders (customer_id, total) VALUES (42, 99.99) RETURNING order_id;

-- Outbox entry (same transaction)
INSERT INTO outbox (event_type, payload)
VALUES ('order.created', '{"order_id": 1, "customer_id": 42, "total": 99.99}');
COMMIT;

-- Separate message relay process reads and publishes outbox events
-- If publish fails → retry; if DB commits → guaranteed at-least-once delivery

```

State Machine Pattern

```

CREATE TABLE orders (
    order_id  BIGINT  GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    status    TEXT    NOT NULL DEFAULT 'draft'
                    CHECK (status IN ('draft','submitted','paid','processing',
                                      'shipped','delivered','cancelled','refunded'))
);

-- Define valid transitions
CREATE TABLE order_status_transitions (
    from_status TEXT NOT NULL,
    to_status   TEXT NOT NULL,
    PRIMARY KEY (from_status, to_status)
);

INSERT INTO order_status_transitions VALUES
('draft',      'submitted'),
('submitted',  'paid'),
('submitted',  'cancelled'),
('paid',       'processing'),
('paid',       'cancelled'),
('processing', 'shipped'),
('shipped',    'delivered'),
('delivered',  'refunded'),
('cancelled',  'draft'); -- can re-open

```

```

-- Function: validates and performs transition
CREATE OR REPLACE FUNCTION transition_order(p_order_id BIGINT, p_new_status TEXT)
RETURNS void LANGUAGE plpgsql AS $$
DECLARE
    v_current_status TEXT;
BEGIN
    SELECT status INTO v_current_status FROM orders WHERE order_id = p_order_id FOR
    UPDATE;

    IF NOT EXISTS (
        SELECT 1 FROM order_status_transitions
        WHERE from_status = v_current_status AND to_status = p_new_status
    ) THEN
        RAISE EXCEPTION 'Invalid transition: % → %', v_current_status, p_new_status;
    END IF;

    UPDATE orders SET status = p_new_status WHERE order_id = p_order_id;

    -- Log the transition
    INSERT INTO order_events (order_id, from_status, to_status) VALUES
        (p_order_id, v_current_status, p_new_status);
END;
$$;

```

Common Mistakes

1. **Soft deletes without partial indexes** — Every query filters `WHERE deleted_at IS NULL` ; without a partial index, PostgreSQL scans all rows including deleted ones.
2. **Using generic entity_type/entity_id** without FK constraints — No referential integrity, no index optimization, impossible to cascade deletes.
3. **Not using AFTER trigger for audit logs** — BEFORE triggers can modify NEW/OLD; use AFTER for reliable audit logging that captures the final committed values.
4. **Storing version numbers in the application** — The version check must happen in the database (atomic compare-and-swap), not the application.
5. **Infinite recursion in CTEs without depth limit** — Always add a depth counter and limit to recursive CTEs on self-referencing tables.

Best Practices

1. **Prefer soft deletes** for any data that has compliance or recovery requirements.
2. **Use generic audit triggers** applied to all important tables consistently.
3. **Use partial indexes** to exclude soft-deleted rows from operational indexes.
4. **Use separate FK columns** for polymorphic associations — never the generic `entity_type/entity_id` pattern.
5. **Use ltree extension** for hierarchical data that requires frequent traversal.

6. **Combine outbox pattern** with change data capture (CDC) via logical replication for high-throughput event sourcing.
 7. **Use version column** for any entity modified by concurrent users.
-

Performance Considerations

```
-- Soft delete: partial index keeps index lean
-- Without: index contains all rows (deleted + active)
-- With:    index contains only active rows
-- 90% of queries are for active records → 10x smaller index, 10x faster lookups

-- Audit log: high write volume
-- Partition by month to keep recent data fast:
CREATE TABLE audit_log_2024_01
    PARTITION OF audit_log
    FOR VALUES FROM ('2024-01-01') TO ('2024-02-01');

-- Hierarchy (ltree): path matching uses GIST index
-- Depth-10 tree with 1M nodes: O(log n) subtree queries vs O(n) recursive CTE
```

Interview Questions & Answers

Q1: What is a soft delete and what are its trade-offs?

A: A soft delete marks rows as deleted (e.g., `deleted_at` timestamp) rather than physically removing them. Trade-offs: Pros: data recovery, audit trail, no FK cascade issues. Cons: all queries must filter `WHERE deleted_at IS NULL`, unique constraints need workarounds (partial unique indexes), table grows indefinitely without archiving.

Q2: What is the difference between an audit log and a history table?

A: An audit log records who changed what and when (one entry per operation, with old/new values as JSONB). A history table maintains full row versions with temporal validity periods (`valid_from/valid_to`), enabling "what was the state at time T" queries. History tables are richer but heavier to maintain.

Q3: Why is the generic `entity_type/entity_id` polymorphic pattern dangerous?

A: No foreign key constraints — you can reference entities that don't exist, and deleting an entity won't cascade to its polymorphic comments/likes/etc. No index optimization. No type checking. The preferred alternative is separate nullable FK columns with a CHECK constraint ensuring exactly one is non-NULL.

Q4: What is optimistic locking and when would you use it?

A: Optimistic locking assumes conflicts are rare. You read data (including a version number), process it, then update with a `WHERE version = :read_version` check. If another process modified it, the update affects 0 rows and you retry. Use it for: document editing, collaborative applications, any scenario with infrequent concurrent writes to the same record.

Q5: What is the outbox pattern and why is it useful?

A: The outbox pattern stores messages/events in a database table in the same transaction as the business operation. A separate relay process reads the outbox and publishes to a message queue. This ensures

atomicity: either both the DB write and the message happen, or neither does. It solves the "dual-write" problem in event-driven architectures.

Q6: How do you prevent infinite loops in recursive CTEs on self-referencing tables?

A: (1) Add a `depth` counter and `WHERE depth <= N` limit. (2) PostgreSQL 14+ has `CYCLE` clause: `WITH RECURSIVE ... CYCLE id SET is_cycle USING path`. (3) Maintain an array of visited IDs and check `id != ALL(path_array)`.

Q7: When would you choose ltree over adjacency list for a hierarchy?

A: Use `ltree` when: queries traverse the hierarchy frequently (subtrees, ancestors, depth levels), the hierarchy is relatively stable (path updates require touching all descendants), and you need $O(\log n)$ subtree queries instead of recursive CTEs. Adjacency list is fine for hierarchies that are mostly written to and rarely traversed deeply.

Q8: How would you implement a state machine in PostgreSQL?

A: Store a `status` column with a CHECK constraint for valid states. Create a `transitions` table with `(from_status, to_status)` pairs. Write a function that validates the transition against the table and performs the update atomically. This is more maintainable and flexible than a large CHECK constraint listing all valid transitions.

Exercises with Solutions

Exercise 1

Add soft delete to the `users` table and update the unique email constraint to allow multiple deleted users with the same email.

Solution:

```
ALTER TABLE users ADD COLUMN deleted_at TIMESTAMPTZ;

-- Drop existing unique constraint on email
ALTER TABLE users DROP CONSTRAINT users_email_key;

-- Replace with partial unique index (only active users must have unique email)
CREATE UNIQUE INDEX uq_users_email_active ON users (email)
    WHERE deleted_at IS NULL;

-- Soft delete function
CREATE OR REPLACE FUNCTION soft_delete_user(p_user_id BIGINT)
RETURNS void LANGUAGE SQL AS $$
    UPDATE users SET deleted_at = now() WHERE user_id = p_user_id AND deleted_at IS
    NULL;
$$;
```

Exercise 2

Create a comprehensive audit system for the `products` table that captures all changes with the application user ID (stored in a session variable).

Solution:

```
CREATE TABLE products_audit (  
    audit_id          BIGINT  GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
    product_id        INTEGER NOT NULL,  
    operation          TEXT    NOT NULL CHECK (operation IN  
( 'INSERT', 'UPDATE', 'DELETE' )),  
    old_data           JSONB,  
    new_data           JSONB,  
    db_user            TEXT     NOT NULL DEFAULT current_user,  
    app_user_id        BIGINT,  -- from session variable  
    changed_at         TIMESTAMPTZ NOT NULL DEFAULT now()  
);  
  
CREATE INDEX idx_products_audit_id  ON products_audit (product_id, changed_at  
DESC);  
CREATE INDEX idx_products_audit_user ON products_audit (app_user_id);  
  
CREATE OR REPLACE FUNCTION audit_products()  
RETURNS TRIGGER LANGUAGE plpgsql AS $$  
BEGIN  
    INSERT INTO products_audit (product_id, operation, old_data, new_data,  
app_user_id)  
    VALUES (  
        COALESCE(NEW.product_id, OLD.product_id),  
        TG_OP,  
        CASE WHEN TG_OP = 'INSERT' THEN NULL ELSE to_jsonb(OLD) END,  
        CASE WHEN TG_OP = 'DELETE' THEN NULL ELSE to_jsonb(NEW) END,  
        NULLIF(current_setting('app.user_id', TRUE), '')::BIGINT  
    );  
    RETURN NULL;  
END;  
$$;  
  
CREATE TRIGGER trg_audit_products  
    AFTER INSERT OR UPDATE OR DELETE ON products  
    FOR EACH ROW EXECUTE FUNCTION audit_products();  
  
-- Usage: SET app.user_id = '42';  
-- Then normal DML operations are automatically audited
```

Cross-References

- [01_normalization_1nf_2nf_3nf.md](#) — normalized base before patterns
- [03_denormalization.md](#) — counter caches (a denormalization pattern)
- [05_schema_design_ecommerce.md](#) — soft deletes and audit in practice
- [06_schema_design_banking.md](#) — immutable ledger as audit trail
- [../09_Transactions_Concurrency/](#) — optimistic locking and MVCC
- [../14_Security/](#) — audit trails for compliance